

Core ML: Long-Context Transformer Experiments

SAiDL Summer Induction Assignment 2026

Neel Joshi

June 2026

Contents

1	What This Is	2
2	Setup	2
3	Part 1: Baseline	2
4	Part 2: Attention Variants	3
4.1	Sliding Window Attention	3
4.2	Linear Attention	3
4.3	Grouped Query Attention (GQA)	3
4.4	Results	3
4.5	What I noticed	3
5	Part 3: Positional Encodings and Extrapolation	4
5.1	The methods	4
5.2	Results	4
5.3	What I noticed	4
6	Part 4: Convolution Hybrids	5
6.1	Conv-before-attention	5
6.2	Gated Conv FFN	5
6.3	What I noticed	6
7	Full Results Table	6
8	Main Takeaways	6

1 What This Is

This report documents experiments I ran for the Core ML compulsory task. The goal was to build a Transformer language model from scratch, make it modular enough that I could swap out individual components, and then actually understand what each component does by changing one thing at a time and measuring what happens.

I trained everything on WikiText-2 — basically cleaned Wikipedia text — with the task of predicting the next token. I used a small model (4 layers, 256-dim, 4 heads, about 16M parameters) so I could run enough experiments on Colab without dying. The metric I tracked is **perplexity** — lower means the model is less surprised by the next word, so lower is better.

One thing I want to note upfront: 10 epochs from random initialisation is nowhere near enough to get state-of-the-art perplexity. My absolute numbers are high (200–650 range). But that wasn't really the point — the point was to compare things fairly, and for that you just need consistent training conditions, which I had.

2 Setup

Model: 4 transformer layers, $d_{\text{model}} = 256$, 4 attention heads, $d_{\text{ff}} = 1024$, dropout 0.1, weight tying between token embedding and LM head. Around 16M parameters depending on the config.

Data: WikiText-2, tokenised with GPT-2's tokeniser (vocab size 50,257). 2.42M training tokens, 249k validation tokens.

Training: AdamW, $\text{lr} = 3 \times 10^{-4}$, cosine schedule with 10% warmup, gradient clipping at 1.0, batch size 16, sequence length 512, 10 epochs. Google Colab T4 GPU.

The key design decision was making everything go through a single `ModelConfig` dataclass. To run a new experiment I just change one field — `attention_type`, `pe_type`, or `conv_type` — and everything else (data loading, training loop, logging) stays identical. This was the only way I could trust that differences in perplexity were actually caused by the component I changed.

3 Part 1: Baseline

Standard Transformer as described in Vaswani et al. (2017) [1]: sinusoidal positional encoding, full softmax self-attention with causal masking, GELU feed-forward, pre-LayerNorm residual connections.

Baseline result: **PPL = 635.99**, 8,013 MB peak GPU memory, 104 seconds per epoch, 64,247 tokens/sec throughput.

The training was still clearly improving at epoch 10 (loss 6.34 train vs 6.46 val) so it hadn't converged. But the gap between train and val is small which at least means no overfitting.

4 Part 2: Attention Variants

I tested three alternatives to full softmax attention, all still using sinusoidal PE so the only variable is the attention mechanism.

4.1 Sliding Window Attention

Each token only attends to the $w = 64$ most recent tokens. Cost drops from $O(T^2)$ to $O(T \cdot w)$. Implemented by masking out scores where $i - j > w$ on top of the standard causal mask.

4.2 Linear Attention

Replaces softmax with a kernel trick: $\phi(Q)(\phi(K)^\top V)$ instead of $\text{softmax}(QK^\top)V$, where $\phi(x) = \text{ELU}(x) + 1$. The key identity is that the order of operations can be rearranged to avoid computing the full $T \times T$ matrix, reducing complexity to $O(T)$.

4.3 Grouped Query Attention (GQA)

Fewer KV heads than query heads. I used $n_{kv} = 2$ with $n_h = 4$, so each KV head is shared by 2 query heads. Main motivation is reducing KV cache memory during inference.

4.4 Results

Table 1: Attention variants. All trained with sinusoidal PE, seq_len=512, 10 epochs.

Method	PPL	Mem (MB)	s/epoch	tok/s
Baseline (standard)	635.99	8,013	104	64,247
Sliding Window	547.15	8,016	107	62,936
Linear Attention	601.43	9,725	154	47,955
GQA ($n_{kv} = 2$)	599.80	8,010	104	65,268

4.5 What I noticed

Sliding window actually beat full attention, which surprised me at first. But it makes sense for Wikipedia — most of what you need to predict the next word is within the current paragraph, not from an article section 400 tokens back. Forcing the model to focus locally turned out to be a useful constraint.

Linear attention was slower and used *more* memory than standard attention, which is the opposite of what you'd expect from the theory. After looking into this, the issue is that at seq_len=512, the per-token outer product state ($d_h \times d_h = 64 \times 64$ per token) dominates over the 512×512 standard attention matrix. The crossover where linear becomes cheaper is probably around seq_len 2048+. So if I had longer sequences the story would flip, but at 512 the theoretical benefit doesn't materialise.

GQA basically matched standard attention quality (599 vs 636) with nearly identical memory, which is exactly what it's supposed to do. The real benefit would show up at inference time with a smaller KV cache, not so much in this training benchmark.

5 Part 3: Positional Encodings and Extrapolation

This was the most interesting part of the whole assignment to me. I tested four positional encoding methods, trained each on `seq_len=512`, and then evaluated them on sequences of length 512, 1024, and 2048 without any retraining.

5.1 The methods

RoPE [2] rotates query and key vectors by a position-dependent angle before computing dot products. The nice property is that the dot product between q_i and k_j ends up depending only on $i - j$ (the relative distance), not on absolute positions. This means positions 513, 514, etc. are just slightly larger rotation angles — not foreign territory.

ALiBi [3] doesn't touch the embeddings at all. It just subtracts a penalty proportional to token distance directly from attention scores: $\text{score}(i, j) - = m_h \cdot |i - j|$. Each head gets a different slope m_h , so some heads naturally attend further and some attend closer. The elegance here is that the penalty extends smoothly to any distance — there's no concept of a "maximum position."

Relative PE learns a bias table indexed by relative distance, added to attention scores. Unlike ALiBi the bias shape is unconstrained and learned, not fixed.

Sinusoidal is the original fixed sine/cosine encoding. Works well at training length, but positions beyond training range are essentially meaningless to the model.

5.2 Results

Table 2: PE comparison at training length. Standard attention, `seq_len=512`.

PE	PPL	Mem (MB)	s/epoch	tok/s
Sinusoidal	605.67	9,912	105	64,242
Relative	273.32	9,921	107	63,207
RoPE	205.44	9,918	108	62,599
ALiBi	206.73	10,181	104	64,558

Table 3: Extrapolation test. Trained on 512 tokens, evaluated on longer sequences.

PE	PPL @ 512	PPL @ 1024	PPL @ 2048
Sinusoidal	605.67	766.28	2,516.35
Relative	273.32	280.82	293.33
RoPE	205.44	219.14	258.15
ALiBi	206.73	204.99	204.18

5.3 What I noticed

The numbers here were genuinely shocking to me. RoPE and ALiBi are roughly **3× better** than sinusoidal at the same training length — same model, same data, same training, just a different formula for encoding position. I did not expect the choice of

positional encoding to dominate everything else by this margin. No attention variant change came anywhere close to this effect.

The extrapolation table makes it even clearer. Sinusoidal goes from 605 at 512 tokens to 2,516 at 2048 — it basically falls apart. The model has never seen position 600 during training, so it has no idea what to do with it.

ALiBi gets *better* with longer sequences (206.73 $\rightarrow$ 204.18). That’s not a typo. With more context tokens available, the model just uses them better. The linear distance penalty naturally handles any length, so 2048 tokens is just 512 tokens with more context rather than something foreign.

RoPE also holds up well — only +52 PPL across 4 $\times$ the training length. This matches what I read in the RoFormer paper about relative position generalisation.

The main takeaway: if I had to give one piece of advice from these experiments, it would be “use RoPE or ALiBi by default.” It’s free in the sense that you’re not adding parameters or changing the architecture, just changing the PE formula, and the improvement is massive.

6 Part 4: Convolution Hybrids

Both experiments here used Sliding Window attention + RoPE (the best individual components from Parts 2 and 3).

6.1 Conv-before-attention

A causal depthwise Conv1D layer before each attention block, to extract local n-gram features:

$$x' = x + \text{GELU}(\text{CausalDepthwiseConv1D}(x, k = 3))$$

Depthwise means one filter per channel (groups = d_{model}), so it’s cheap — only 4,096 extra parameters total across all layers.

The causal part matters a lot. My first implementation used symmetric padding (padding both sides of the sequence), which let the conv kernel at position t see token $t + 1$. The model immediately cheated by just reading the next token from the conv output — perplexity dropped to 1.3 after 3 epochs, which is physically impossible for legitimate prediction. Validation loss was lower than training loss, which was the giveaway. Fixed by padding only on the left.

Result: PPL = 218.71, 8,081 MB, 108 s/epoch, 61,785 tok/s.

6.2 Gated Conv FFN

Replaces the standard MLP feedforward with:

$$\text{output} = \text{proj}(\text{CausalConv1D}(W_v x) \cdot \sigma(\text{CausalConv1D}(W_g x)))$$

The sigmoid gate learns how much of the conv output to pass through. Unfortunately I hit Colab compute limits before this experiment finished training. Rather than report partial numbers and risk misrepresenting the result, I’ve left it out. The architecture is implemented and the causal fix was applied, but the full 10-epoch run didn’t complete.

6.3 What I noticed

Conv-before-attention hitting 218.71 confirmed the intuition that convolutions and attention are genuinely complementary. Convolutions are good at fixed local patterns (common phrases, punctuation after certain words). Attention is good at variable long-range dependencies. Having the conv pre-process local features before attention uses them seems to help.

7 Full Results Table

Table 4: All completed experiments, sorted by perplexity.

Experiment	Attn	PE	Conv	PPL	Mem (MB)	s/ep	tok/s
PE: RoPE	standard	rope	none	205.44	9,918	108	62,599
PE: ALiBi	standard	alibi	none	206.73	10,181	104	64,558
Conv: Before-Attn	sliding_w	rope	conv_before	218.71	8,081	108	61,785
PE: Relative	standard	relative	none	273.32	9,921	107	63,207
Attn: Sliding Win	sliding_w	sinusoidal	none	547.15	8,016	107	62,936
Attn: Linear	linear	sinusoidal	none	601.43	9,725	154	47,955
Attn: GQA	gqa	sinusoidal	none	599.80	8,010	104	65,268
PE: Sinusoidal	standard	sinusoidal	none	605.67	9,912	105	64,242
Baseline	standard	sinusoidal	none	635.99	8,013	104	64,247

8 Main Takeaways

Three things stand out clearly from these experiments:

- 1. Positional encoding matters more than attention architecture.** The gap between sinusoidal and RoPE/ALiBi (430 PPL) is larger than the gap between the best and worst attention variants (90 PPL). This was unintuitive to me before running the experiments. I'd spent more time reading about efficient attention mechanisms and less about PE, but the data says PE choice has the bigger impact on this benchmark.
- 2. The efficiency claims for linear attention only hold at long sequences.** At `seq_len=512`, linear attention is actually more expensive than standard attention. It only saves memory when the sequence is long enough that the $T \times T$ attention matrix outweighs the per-token state cost. This is a practical detail that doesn't always come through clearly in the papers.
- 3. Causal correctness matters a lot for any local operation.** The conv leakage bug (symmetric padding seeing future tokens) was an important lesson. The attention module has a causal mask built in, so you don't think about it. But any other layer that processes the sequence — conv, pooling, anything — needs explicit causality enforcement. The bug was immediately visible from the perplexity numbers, which is a good argument for always sanity-checking your outputs against theoretical limits.

References

- [1] A. Vaswani et al., “Attention Is All You Need,” *NeurIPS*, 2017.
- [2] J. Su et al., “RoFormer: Enhanced Transformer with Rotary Position Embedding,” *Neurocomputing*, 2024.
- [3] O. Press, N. Smith, M. Lewis, “Train Short, Test Long: Attention with Linear Biases Enables Input Length Extrapolation,” *ICLR*, 2022.
- [4] A. Katharopoulos et al., “Transformers are RNNs: Fast Autoregressive Transformers with Linear Attention,” *ICML*, 2020.
- [5] J. Ainslie et al., “GQA: Training Generalized Multi-Query Transformer Models from Multi-Head Checkpoints,” *EMNLP*, 2023.